DEVOPS CAPSTONE PROJECT

Expense Tracker

From git push to a monitored Kubernetes cluster: tests, security gates, containers, CI/CD, Terraform, Helm, Argo CD and Prometheus + Grafana.

Om Malviya

Roll No 24BCS10448

github.com/om-malviya/expense-tracker-devops



OVERVIEW

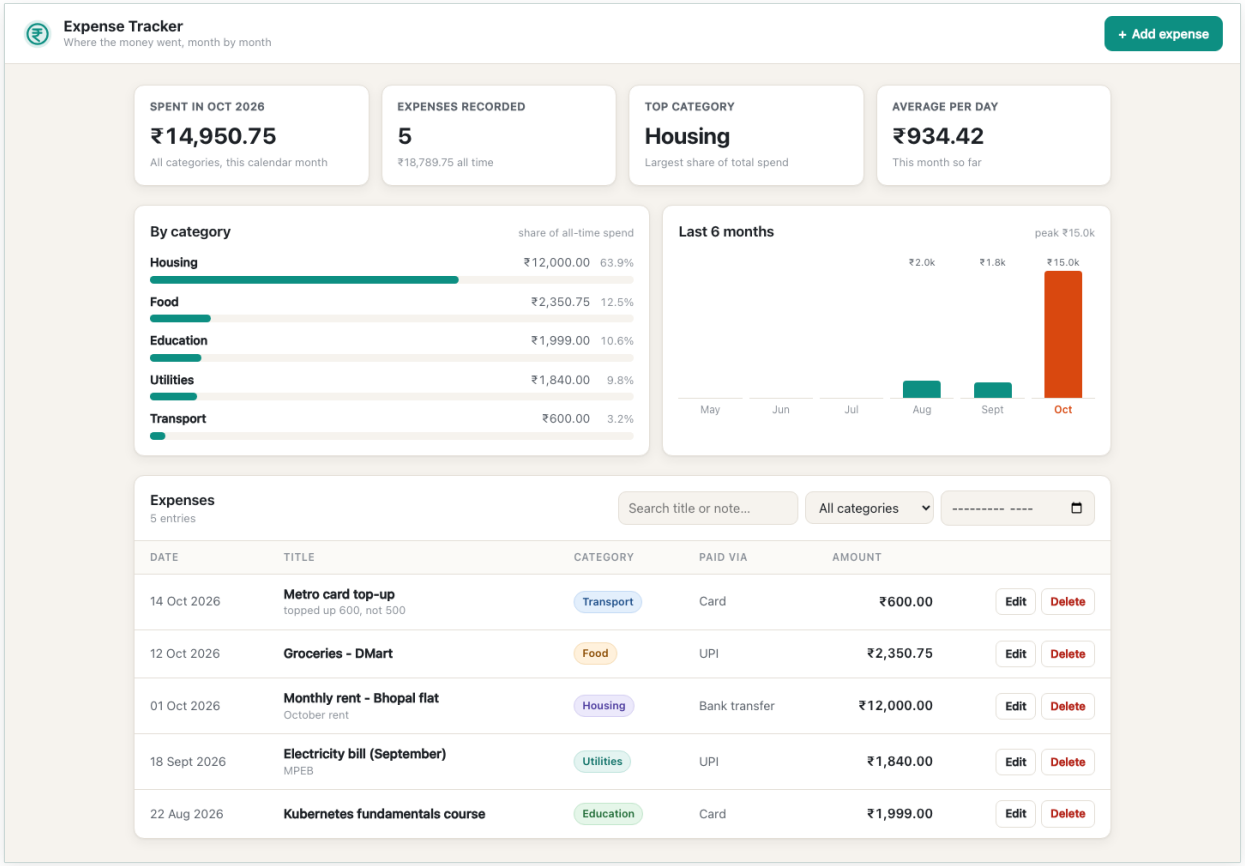
The problem and what I built

The problem

At month-end I want to know how much I spent, on what, and how it compares with earlier months.

What I built

- FastAPI REST API: CRUD, filters, summary
- PostgreSQL 16, schema managed by Alembic
- React dashboard: KPIs, categories, 6-month trend
- Filter by category, month or free text
- Add, edit and delete with validated forms
- Responsive, with loading and error states



Dashboard at localhost:3000, served by nginx from the Docker Compose stack

TECH STACK

One tool per job, from code to cluster



App

FastAPI · React 18 · Vite 7 · served by nginx



Data

PostgreSQL 16 · SQLAlchemy 2 · Alembic migrations



Quality & Security

pytest · ruff · Bandit · pip-audit · Gitleaks · Trivy



Containers

Docker multi-stage builds · Compose · non-root images



CI/CD

GitHub Actions · GHCR images tagged with the git SHA



IaC

Terraform 1.16 · AWS VPC + EKS 1.31 (ap-south-1)



Orchestration

Kubernetes (k3s, EKS) · Helm · Traefik Ingress · HPA



Observability

Prometheus · Grafana · Alertmanager · ServiceMonitor

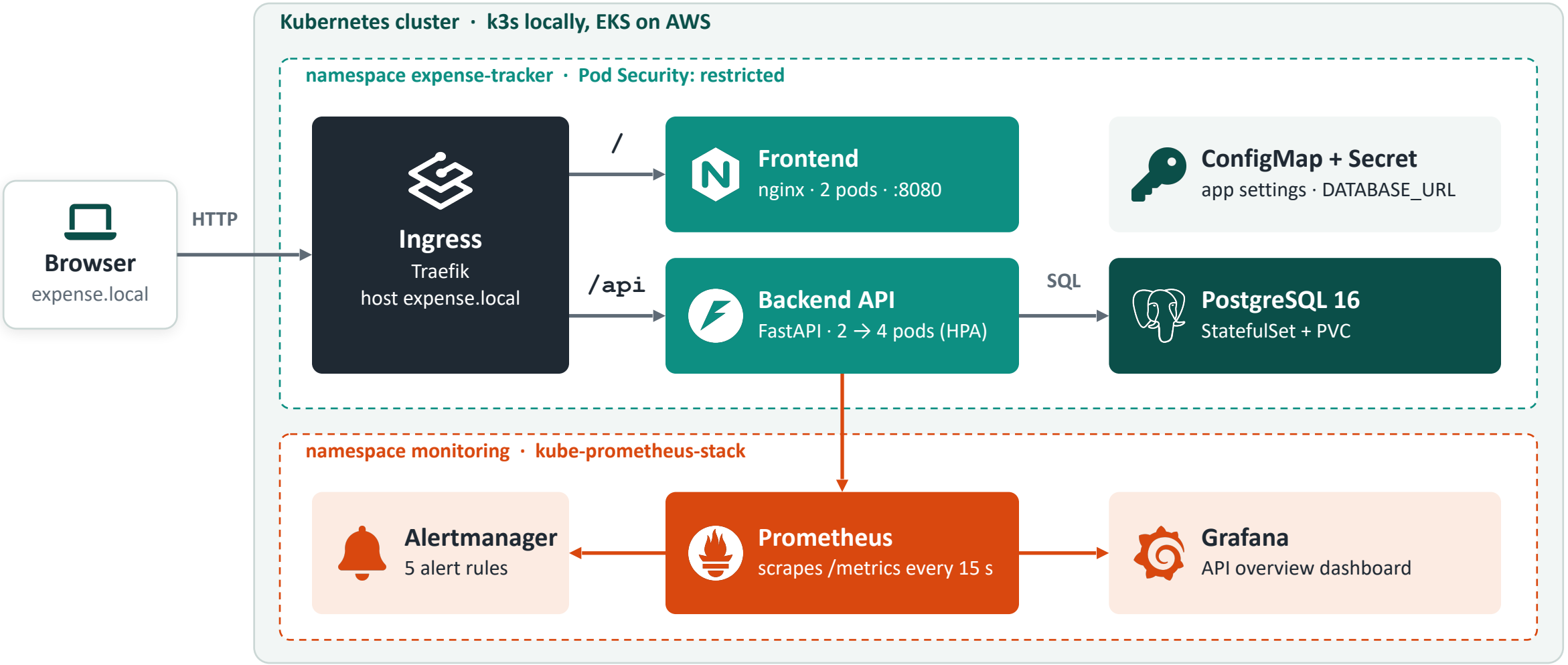


GitOps

Argo CD: automated sync, prune, self-heal

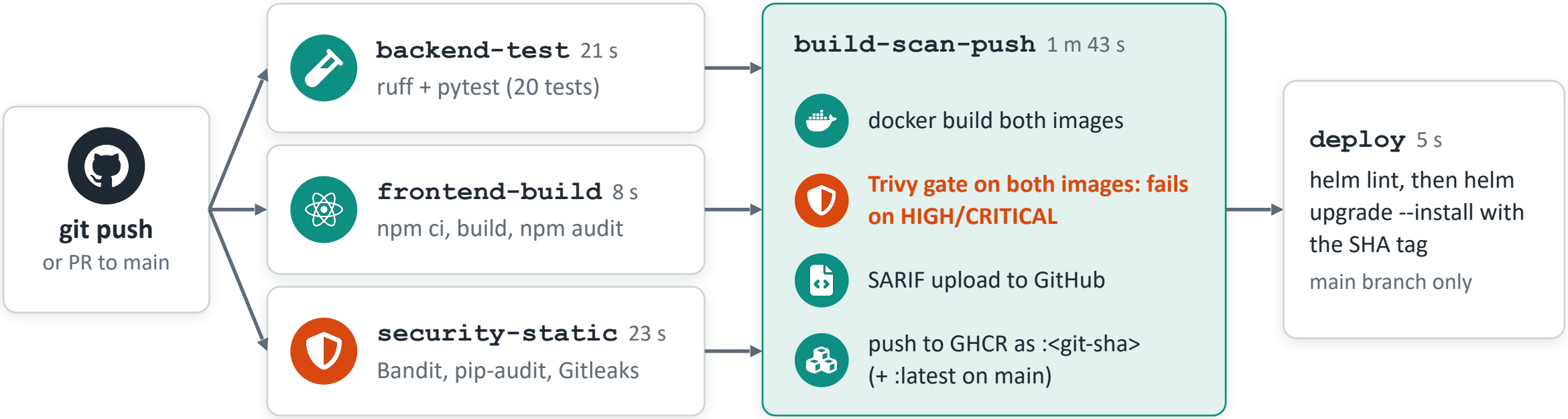
ARCHITECTURE

How a request flows through the cluster



CI/CD

GitHub Actions: five jobs, one security gate



What actually happened on the first push

Run 1 failed after 29 s
Gitleaks: my own DB-URL rule matched a README example

Fix in commit ec416e2
Placeholders in the docs + allow-list; the rule stays on

Run 2 green in 2 m 21 s
All 5 jobs passed; images pushed with the SHA tag

DEVSECOPS

Security checks at every stage

0

HIGH / CRITICAL fixable CVEs

in both images at the Trivy gate

Backend: Debian 13.7, 87 OS + 30 Python packages

Frontend: Alpine 3.24.2, 70 OS packages

A fixable HIGH/CRITICAL CVE turns the job red and nothing reaches GHCR.



Gates in the pipeline

Gitleaks
secrets, scanned over full git history

Bandit
Python static analysis (SAST)

pip-audit --strict
known CVEs in Python dependencies

npm audit
JS dependencies, fails on HIGH+

Trivy
both images, blocks the push



Runtime hardening

Non-root users
uid 10001 (API) and 101 (nginx)

pip removed
not present in the runtime image

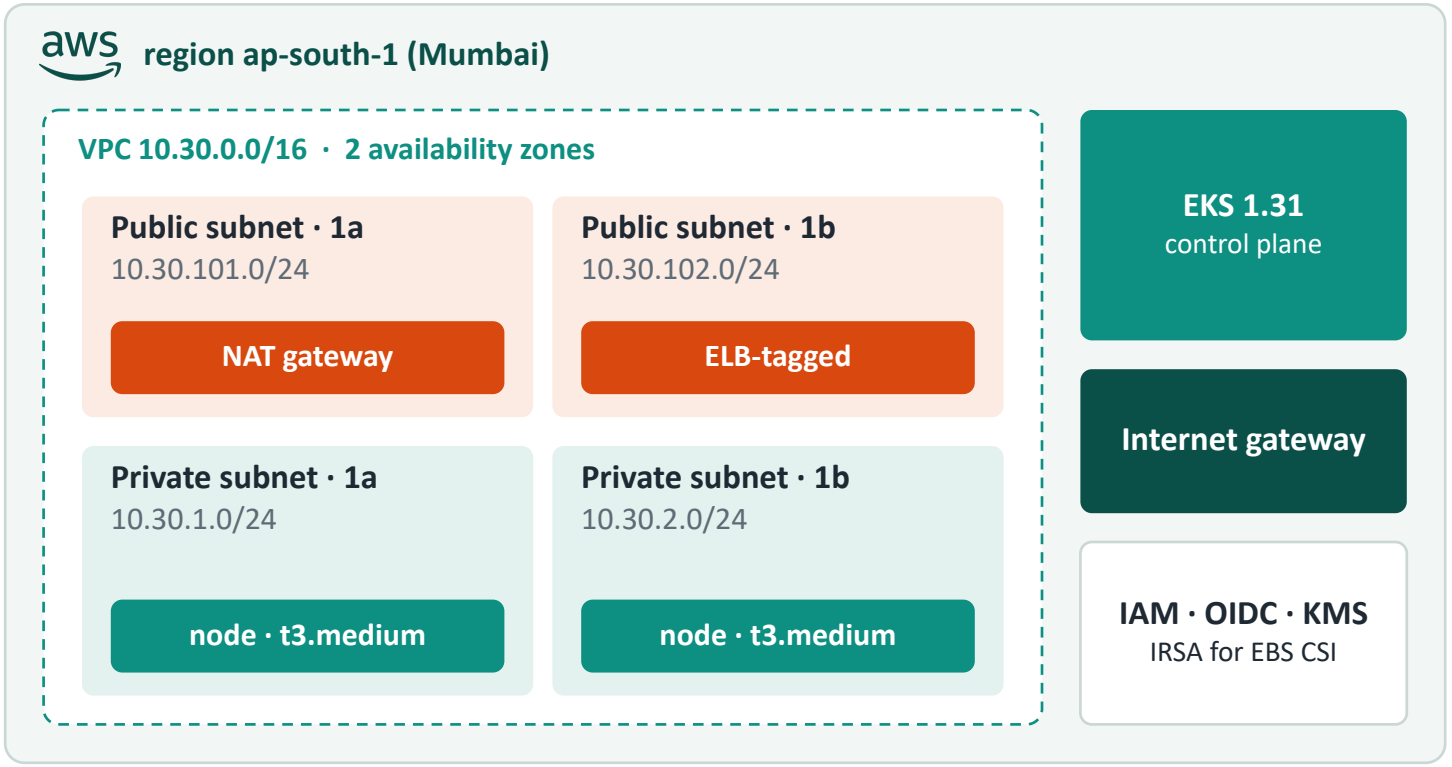
Read-only root filesystem
emptyDirs only where writes are needed

Drop ALL capabilities
no privilege escalation, seccomp default

Pod Security: restricted
enforced on the namespace

INFRASTRUCTURE AS CODE

Terraform: AWS VPC + EKS in ap-south-1



61 **resources to add**

terraform plan
0 to change, 0 to destroy

init, fmt and validate pass; plan ran offline against LocalStack

Cost control

- One NAT gateway shared by both AZs
- t3.medium nodes: min 1, desired 2, max 3
- ~5–6 USD/day: destroy after each demo

init → fmt + validate → plan → apply → check console → destroy

KUBERNETES + HELM

One Helm chart, deployed by git SHA

Deployments
backend + frontend, 2 replicas each, probes

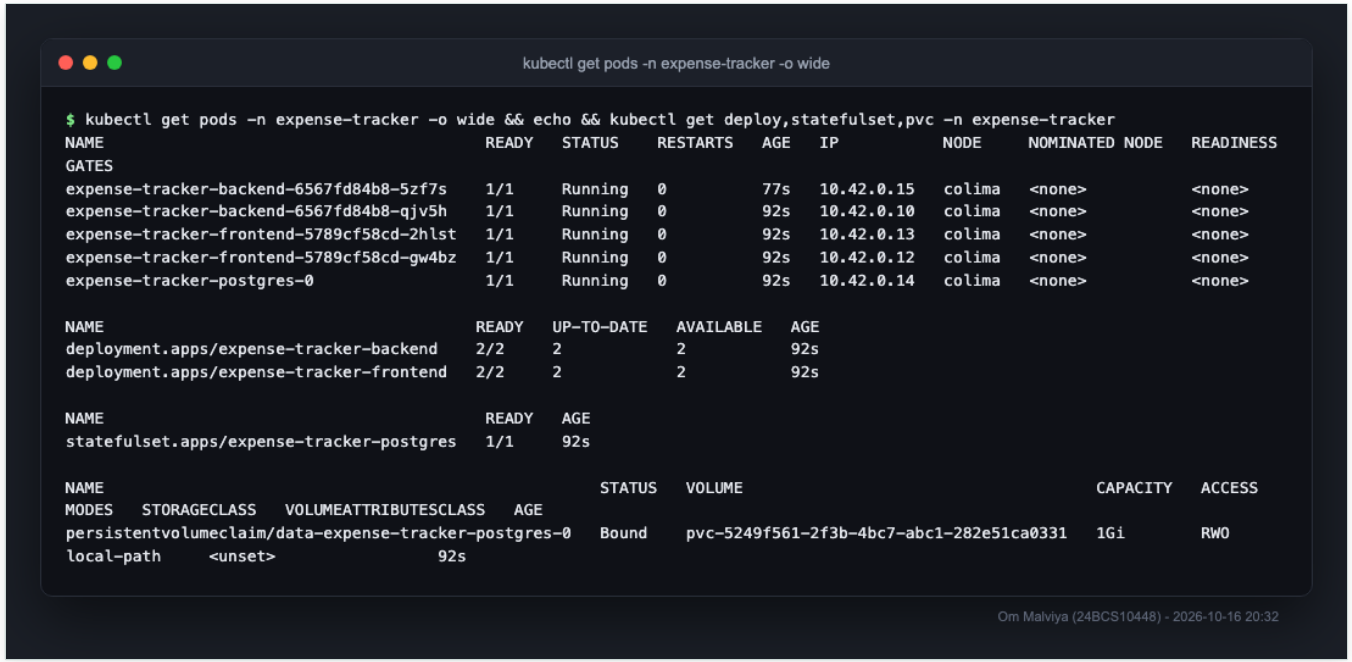
StatefulSet
PostgreSQL 16 with a 1 Gi PVC

ClusterIP Services
backend :8000, frontend :80, postgres

Ingress
expense.local: /api → backend, / → frontend

HPA
backend 2 → 4 pods at 60 % CPU

ConfigMap · Secret · ServiceMonitor
config, DATABASE_URL, metrics scrape



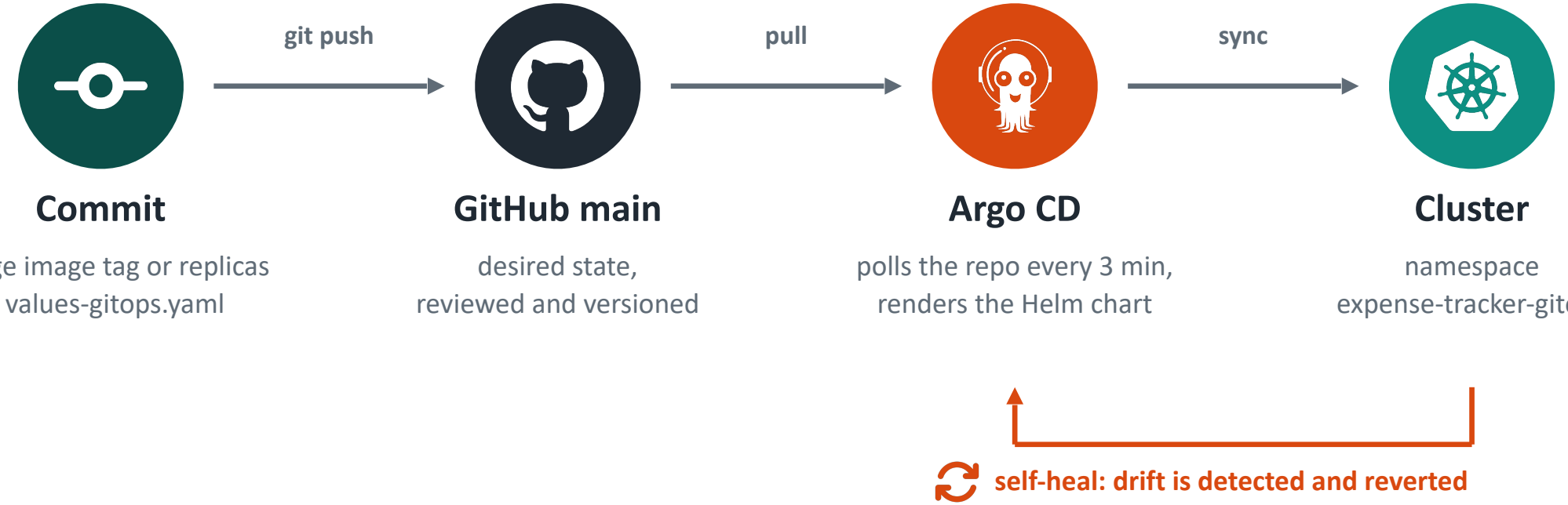
2 → 4
backend pods under load

528 %
peak CPU vs 60 % target

9,000
requests in 180 s, 0 failed

GITOPS

Argo CD: Git is the source of truth



Two delivery paths

Push: the CI deploy job runs helm upgrade with the SHA tag

Pull: Argo CD inside the cluster follows main, no kubeconfig in CI

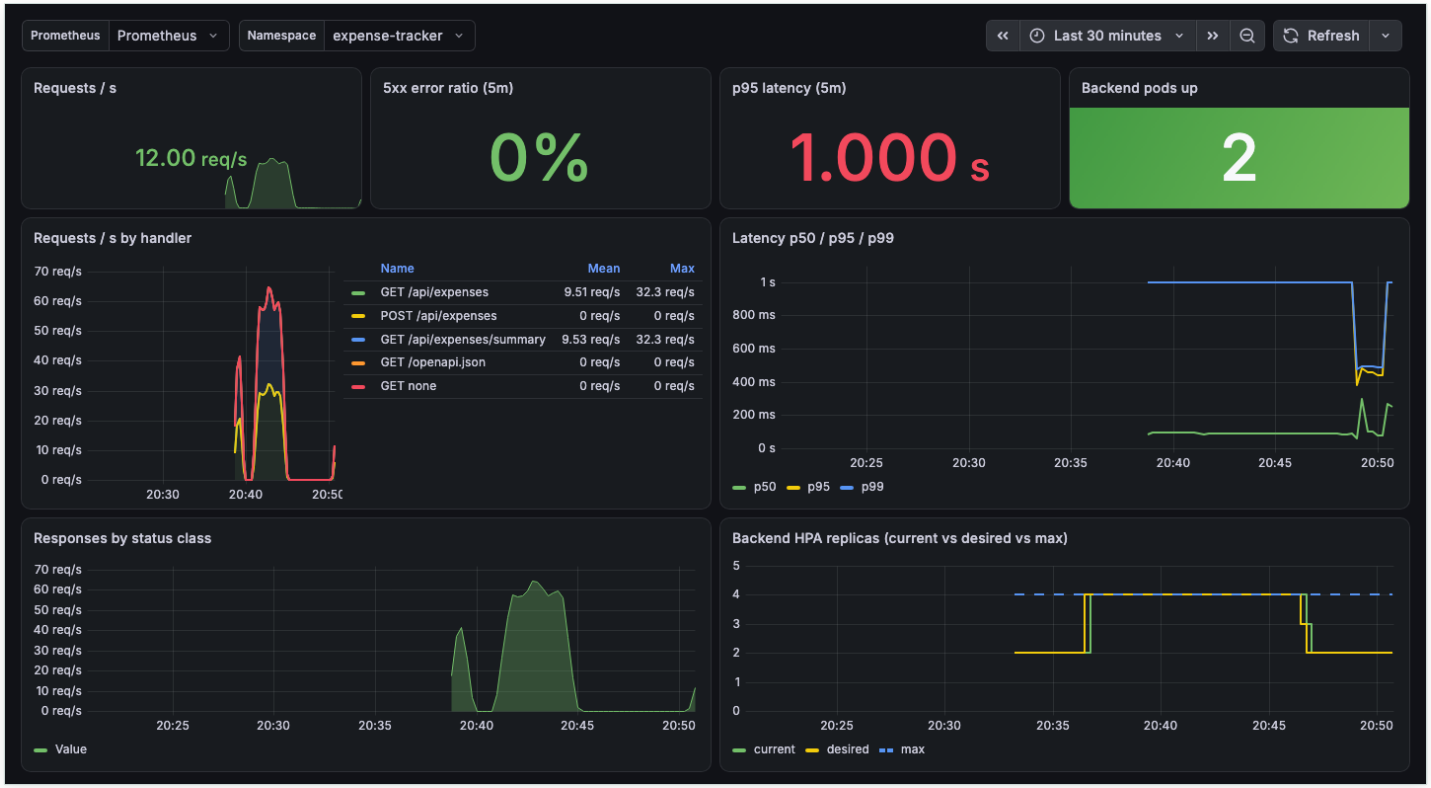
Sync policy: `automated` · `prune` · `selfHeal`

A deployment is a commit; a rollback is a git revert

kubectl scale or a deleted pod is put back to match Git

OBSERVABILITY

Prometheus + Grafana on live app metrics



16
metric families

15 s
scrape interval

8
Grafana panels

0 %
5xx under load

Targets: UP

ServiceMonitor scrapes every backend pod

PromQL I used

```
up{namespace="expense-tracker"}
sum(rate(http_requests_total[5m]))
histogram_quantile(0.95, …)
```

5xx ratio · HPA replicas

5 alert rules

BackendDown · CrashLooping · HighErrorRate · HighLatencyP95 · HpaAtMax

WRAP-UP

Key learnings

1 Scanners read your docs too

Run 1 failed: my Gitleaks rule matched a README example. I fixed the docs and kept the rule.

2 Config keys can fail silently

ignore-unfixed at the top level of trivy.yaml did nothing (44 HIGH, exit 1); it goes under vulnerability.

3 pip is attack surface

Removed pip, setuptools and wheel from the runtime image, and upgraded the nginx base to 1.30.

4 Monitors select Services

My ServiceMonitor used pod labels and found no target; Helm revision 2 fixed the selector.

5 PSA restricted is a free test

It forced runAsNonRoot, seccomp and drop ALL on every container, Postgres included.

6 HPA needs requests + patience

~1.5 min of FailedGetResourceMetric at first; never set replicas while the HPA owns them.

Thank you

Questions?

Om Malviya

Roll No 24BCS10448

github.com/om-malviya/expense-tracker-devops